Merge sort complexity analysis

Divide step (find middle and split) takes $O(n)$

Recur step (solve left and right subproblem) takes 2 $T(n/2)$

Conquer step (merge subarrays) takes $O(n)$

Now we can set up the recurrence for $T(n)$:

$$T(n) = \begin{cases} 2\,T(n/2) + O(n) & \text{for } n > 1 \\ O(1) & \text{for } n = 1 \end{cases}$$

This solves to $T(n) = O(n \log n)$

Solving recurrences by unrolling

General strategy:

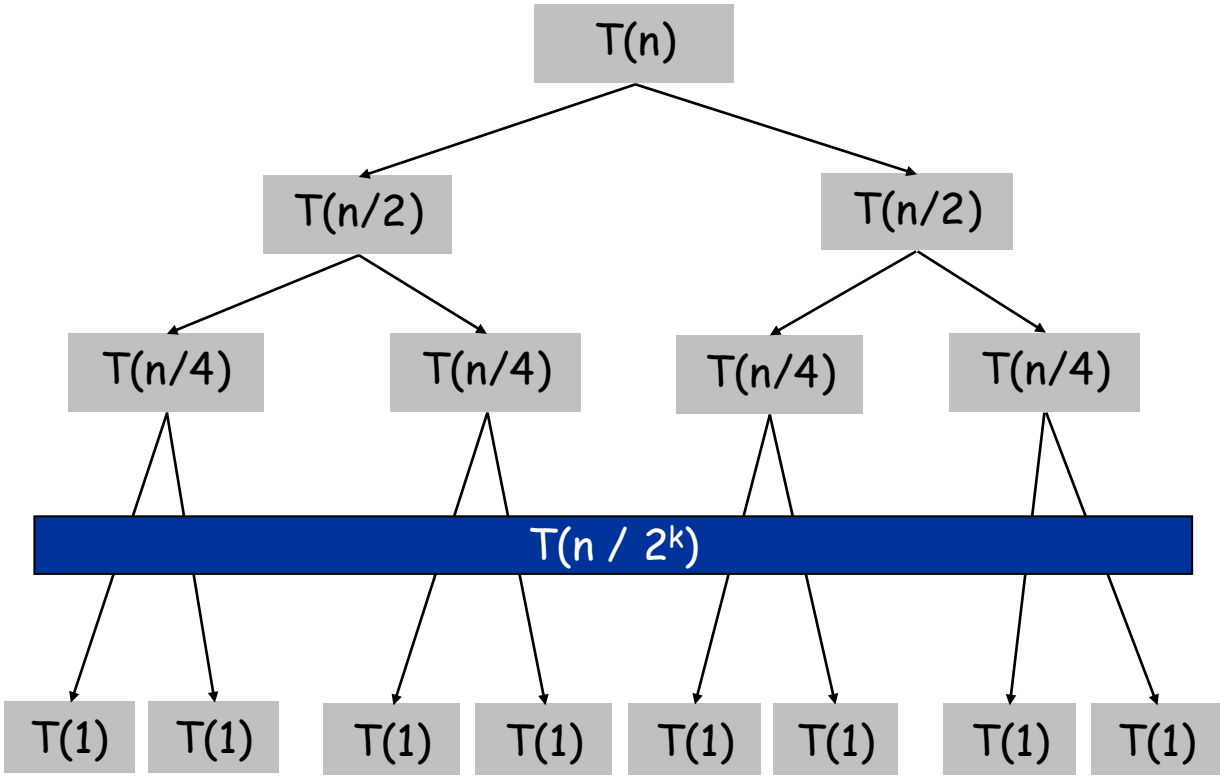
- Analyze first few levels
- Identify the pattern for a generic level
- Sum up over all levels

To verify the solution, we can substitute guess into the recurrence and prove it formally using induction

For Merge sort this method yields $T(n) = O(n \log n)$

There is a "Master theorem" (see textbook) that can handle most recurrences of interest, but unrolling is enough for our purposes

Proof by unrolling: T(n) = 2 T(n/2) + O(n)



1 (of size n)

2 (of size n/2)

4 (of size n/4)

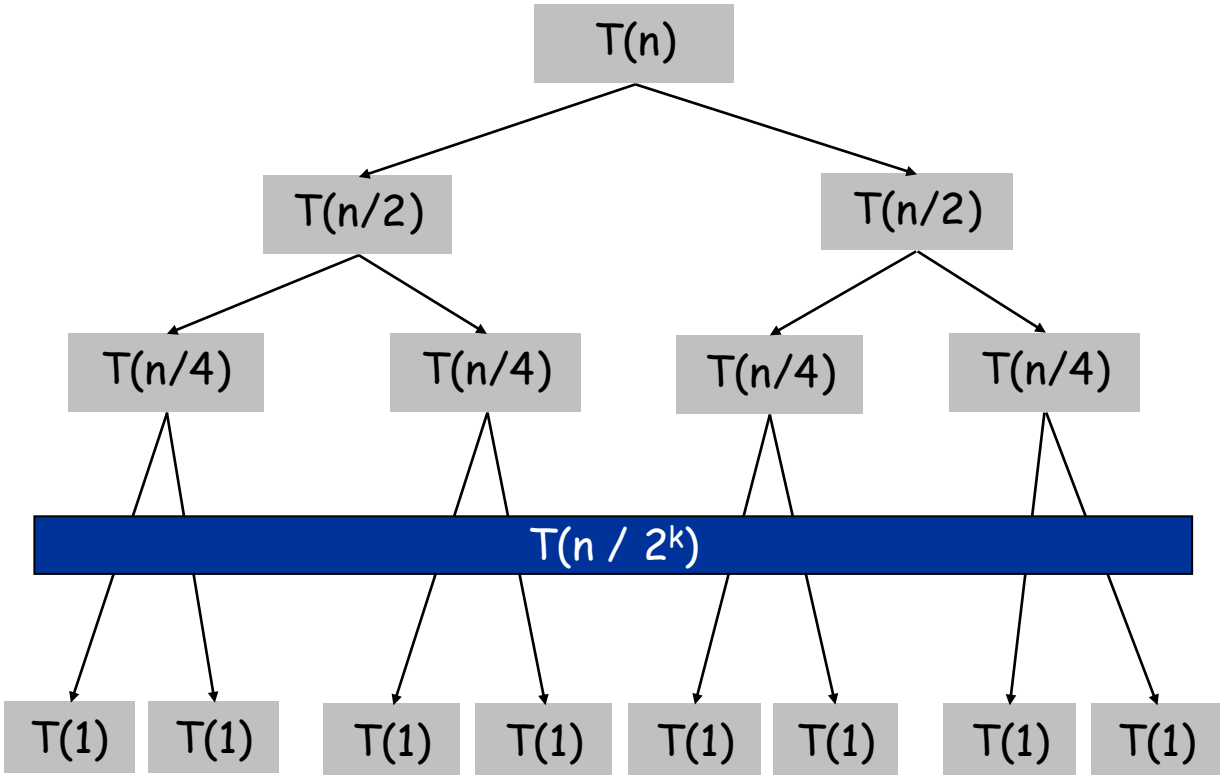
. . .

2^k (of size $n/2^k$)

. . .

n (of size 1)

Proof by unrolling: T(n) = 2 T(n/2) + O(n)



1 (of size n) → cn

2 (of size n/2) → cn

4 (of size n/4) → cn

$\log_2 n$

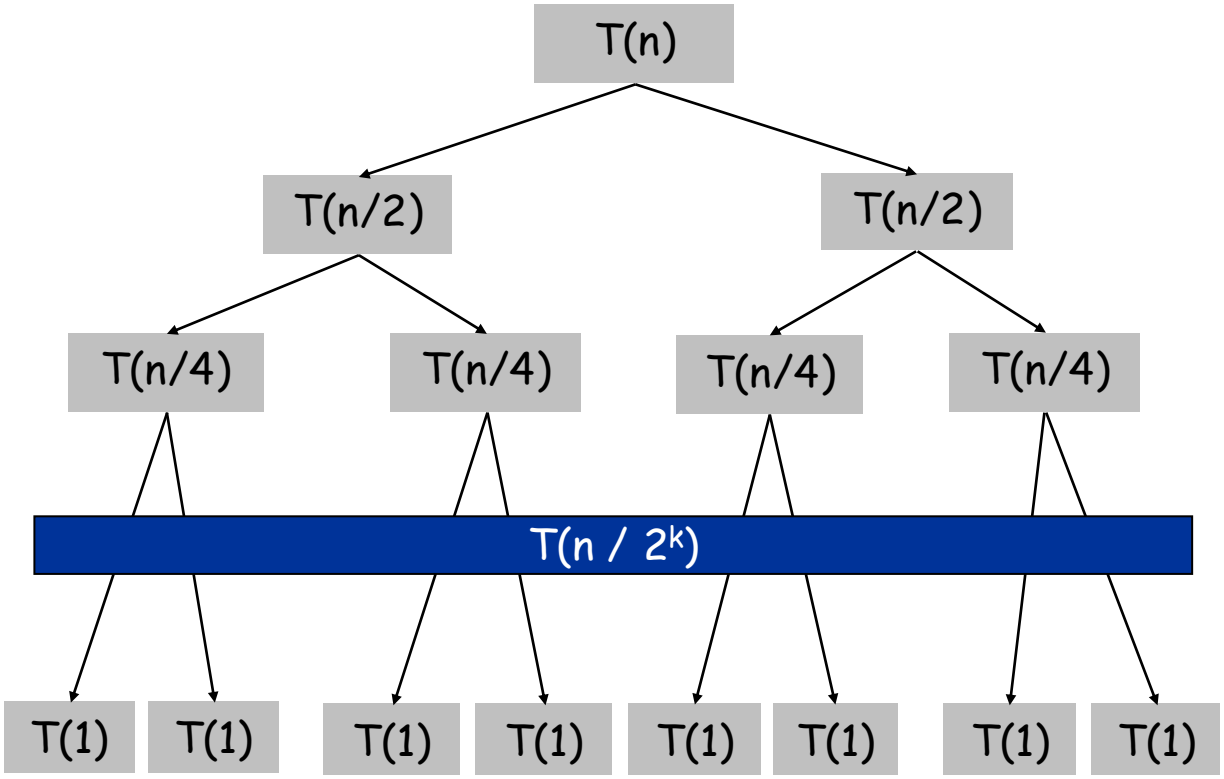
. . .

2^k (of size $n/2^k$) → cn

. . .

n (of size 1) → cn

Proof by unrolling: T(n) = 2 T(n/2) + O(n)



1 (of size n) → cn

2 (of size n/2) → cn

4 (of size n/4) → cn

log n

. . .

2^k (of size $n/2^k$) → cn

. . .

n (of size 1) → cn

cn log n

Some recurrence formulas with solutions

Recurrence	Solution
$T(n) = 2\,T(n/2) + O(n)$	$T(n) = O(n \log n)$
$T(n) = 2\,T(n/2) + O(\log n)$	$T(n) = O(n)$
$T(n) = 2\,T(n/2) + O(1)$	$T(n) = O(n)$
$T(n) = T(n/2) + O(n)$	$T(n) = O(n)$
$T(n) = T(n/2) + O(1)$	$T(n) = O(\log n)$
$T(n) = T(n-1) + O(n)$	$T(n) = O(n^2)$
$T(n) = T(n-1) + O(1)$	$T(n) = O(n)$